

Evaluación Parcial 2: Casos diseñados y suite automatizada

ASIGNATURA	PRO402 · Taller de Testing y Calidad de Software
UNIDAD	02 · Desarrollo y Ejecución de Casos de Prueba
MODALIDAD	Práctica individual · Proyecto incremental
DOCENTE	Diego Obando

1 Sentido de la evaluación

Esta es la segunda entrega del mismo proyecto. En la primera demostraste que tenía una línea base: controles estáticos en verde y pruebas que documentan lo que hace. Ahora hay que demostrar algo distinto y más difícil: que las pruebas que existen **no están ahí por intuición**.

La diferencia entre un estudiante y un profesional del testing no es cuántas pruebas escribe, sino si puede explicar por qué escribió esas y no otras. Esa explicación tiene nombre y método: partición de equivalencia, análisis de valores límite, tablas de decisión. Un caso de prueba derivado con una técnica se puede defender; uno que se le ocurrió a alguien, no.

Lo segundo que se evalúa es que la suite ya no viva en un solo nivel. Un sistema real falla en tres lugares distintos —dentro de una función, en la conversación entre componentes, y en el recorrido completo de una persona usándolo— y cada uno necesita su propia clase de prueba.

2 El proyecto sobre el que se trabaja

El mismo de la EP1, con dos cambios de alcance:

- El sistema debe exponer una **interfaz consumible**: una API con FastAPI, o una interfaz web que se pueda recorrer. Es lo que hace posible probar más allá de la función suelta.
- Las reglas de negocio de la EP1 siguen siendo el centro. Pueden crecer, pero no se reemplazan: lo que se evalúa es cómo evolucionó **este** proyecto, no uno nuevo.

La línea base de la EP1 —entorno reproducible, controles estáticos en verde y pruebas que documentan el comportamiento— se da por supuesta: es el piso sobre el que se construye todo lo que sigue.

El entorno tampoco cambia. `uv`, `ruff`, `pyrefly` y `pytest` siguen siendo el stack, con la misma versión de Python fijada en `.python-version`. Esta entrega suma **FastAPI** para la interfaz consumible y **Playwright** para las pruebas de extremo a extremo, declaradas como dependencias del proyecto igual que las anteriores.

3 Requisitos mínimos para que la entrega sea evaluable

A. Casos derivados con técnicas formales

- Para cada regla de negocio debe existir un **diseño de casos documentado**, no solo el código de las pruebas: qué particiones identificaste, qué valores límite salen de ellas y, cuando la regla combina condiciones, la tabla de decisión correspondiente.
- Cada caso diseñado debe poder rastrearse hasta la prueba que lo implementa, y cada prueba hasta el caso que la origina.
- Si decidiste **no** implementar un caso que la técnica produjo, dilo y explica por qué. Un caso descartado con argumento vale más que uno implementado sin criterio.

B. Suite automatizada en tres niveles

- **Unitarias** con `pytest` sobre las reglas de negocio.
- **De integración** sobre la interfaz consumible: la API respondiendo de verdad, con sus fixtures, su preparación de datos y sus dobles de prueba donde correspondan.
- **Extremo a extremo** con Playwright, recorriendo al menos un flujo completo de uso.
- Los tres niveles se ejecutan con un comando documentado y terminan en verde.
- Cada nivel debe estar probando algo que los otros no pueden detectar. Tres copias de la misma verificación en tres tecnologías distintas no son una pirámide.

C. Plan de pruebas según ISO/IEC/IEEE 29119

Un documento `PLAN-DE-PRUEBAS.md` en el repositorio, con:

1. **Alcance:** qué entra y qué queda explícitamente fuera.
2. **Riesgos del producto** y su prioridad, conectados con las características de calidad que priorizaste en la EP1.

3. **Estrategia:** qué se prueba en cada nivel y por qué en ese y no en otro.
4. **Trazabilidad:** riesgo → requisito → caso de prueba → prueba automatizada. Continúa la tabla que empezaste en la EP1, ahora con los casos derivados.
5. **Criterios de entrada y salida:** qué tiene que cumplirse para considerar la suite confiable.

D. Datos de prueba y protección de datos personales

- Los datos usados en pruebas deben ser **sintéticos**. Si el proyecto maneja datos personales, no puede haber datos reales de personas en el repositorio ni en los ambientes de prueba.
- Debe quedar documentado qué datos usa cada nivel de prueba, cómo se generan y cómo se eliminan al terminar.
- Aplica el principio de minimización: si una prueba no necesita un campo, ese campo no va.

E. Registro del trabajo con agentes

En la sección **Uso de IA o agentes**, además de lo pedido en la EP1, debe quedar registrado al menos un caso donde **usaste un agente para ampliar un conjunto de casos de prueba** y luego lo auditaste: qué casos propuso, cuáles aceptaste, cuáles descartaste y por qué. Interesa especialmente algún caso que el agente haya propuesto y que estuviera mal fundado.

4

Entregables

1. Enlace al repositorio de GitHub.
2. **README.md** actualizado, con los comandos para ejecutar cada nivel de la suite.
3. **PLAN-DE-PRUEBAS.md**.
4. **DISEÑO-DE-CASOS.md** con las particiones, los valores límite y las tablas de decisión.
5. **CALIDAD.md** actualizado desde la EP1.
6. Código fuente y la suite completa en sus tres niveles.

5 Qué se evalúa y cuánto pesa

Criterio	Peso
Diseño de casos con técnicas formales — corrección de las particiones y los límites, pertinencia de las tablas de decisión, y calidad del argumento sobre los casos descartados	25%
Suite automatizada en tres niveles — que cada nivel pruebe lo que le corresponde y detecte lo que los otros no	25%
Plan de pruebas y trazabilidad — coherencia entre riesgos, estrategia y lo que la suite realmente ejecuta	20%
Datos de prueba y protección de datos — datos sintéticos, minimización, aislamiento y eliminación verificable	10%
Evolución respecto de la EP1 — que se vea el mismo proyecto volviéndose más confiable, con las observaciones de la primera entrega resueltas	10%
Criterio frente al agente — la auditoría de los casos que propuso y las decisiones que tomaste tú	10%

6 Verificación en vivo

En la sesión de evaluación, sobre tu repositorio ya entregado, se introducen **tres defectos distintos**, uno por nivel de la pirámide:

1. Un cambio en una regla de negocio, que solo una prueba unitaria debería detectar.
2. Un cambio en el contrato entre la interfaz y la lógica —un campo renombrado, un código de respuesta cambiado—, que solo una prueba de integración debería detectar.
3. Un cambio en el flujo de uso —un botón que deja de aparecer, un paso que se salta—, que solo una prueba extremo a extremo debería detectar.

Se ejecuta la suite completa y se observa cuáles de los tres se atrapan y en qué nivel. Esta verificación alimenta el criterio de suite automatizada.

Cómo se eligen los cambios

Los tres los genera un **agente de IA** —Claude Code o Codex, el mismo para toda la cohorte— sobre el clon de tu repositorio, con las mismas reglas de la EP1: cada cambio sale de un catálogo fijo, toca un solo lugar y se aplica sobre algo que tú declaraste. Lo que cambia en esta entrega es que cada uno apunta a un nivel distinto de la pirámide:

- el primero, sobre una regla de negocio de tu `README.md`;
- el segundo, sobre el contrato de tu interfaz: un campo renombrado, un código de respuesta cambiado, un parámetro que pasa a ser opcional;
- el tercero, sobre el flujo que recorre tu prueba de extremo a extremo.

Los tres cambios se te muestran antes de ejecutar la suite y quedan registrados junto con el resultado.

Rige la misma impugnación que en la EP1, con las dos causales de siempre —que el cambio no altere el comportamiento observable, o que toque código fuera de lo que declaraste— y se aplica a cada uno de los tres por separado.

7 Qué baja la evaluación

- Pruebas sin caso de prueba que las origine, o casos diseñados que no existen en la suite.
- Particiones que no particionan nada: rangos que se solapan, o una sola clase de equivalencia por regla cuando la regla tiene varias.
- Valores límite mal calculados, o probados solo por un lado del borde.
- Pruebas de integración que en realidad son unitarias con otro nombre, o pruebas extremo a extremo que no recorren un flujo real.
- Un plan de pruebas que describe una estrategia que la suite no ejecuta.
- Datos personales reales en el repositorio, en los fixtures o en las capturas.
- Pruebas inestables entregadas como si estuvieran en verde: si algo falla intermitentemente, hay que decirlo.
- Casos generados por un agente e incorporados sin auditar.
- No poder explicar de qué técnica salió un caso de tu propia entrega.

8 Política de IA y agentes

Rige lo mismo que en la EP1, con un agregado propio de esta entrega.

Generar casos de prueba es lo que un agente hace mejor y peor al mismo tiempo: produce muchos, muy rápido, y sin distinguir los que importan de los que no. Por eso acá el uso está no solo permitido sino esperado, y lo que se evalúa es la auditoría.

Si un caso de prueba viene de un agente, tienes que poder decir **de qué técnica se deriva y qué partición o límite cubre**. Un caso que no se puede ubicar en el diseño no cuenta, aunque la prueba pase.

9 Método de entrega

1. Sube el proyecto a GitHub, en el mismo repositorio de la EP1.
2. Verifica que sea público o esté compartido con el docente.
3. Confirma que el `README.md` permite ejecutar los tres niveles de la suite.
4. Envía el enlace por el canal oficial de la asignatura.

La verificación en vivo se ejecuta sobre el estado que tenga el repositorio en ese momento.